Hybrid Parallelism & Parallelism for Distributed Deep Learning

CS - 598 : Systems for Gen-AI

Presenters: Porter Shawver, Shreyash Bhatkar, Haochen Tong & Runying Chen

Instructor: Prof. Fan Lai

Date: 11th February, 2026.

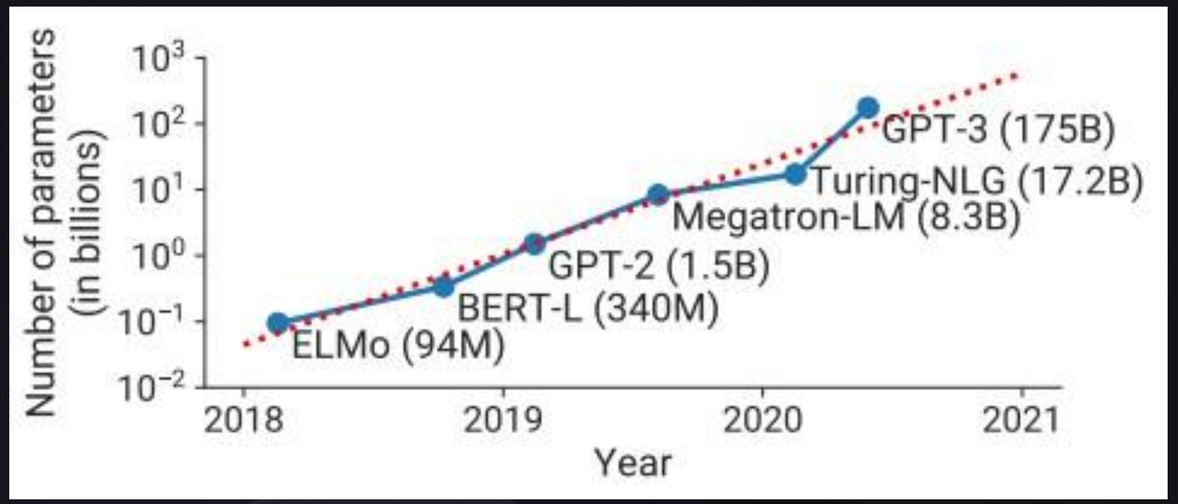
Agenda for today’s Presentation:

Dive into Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM

- Motivation: Why scaling large language models is hard
- Parallelism fundamentals: Data, tensor, and pipeline parallelism
- Pipeline inefficiencies: Microbatching and pipeline bubbles
- Megatron-LM’s hybrid parallelism strategy
- Optimization techniques and performance results
- Limitations, discussion, and future directions

The Scale Revolution in Language Models

- Language models are growing exponentially
- GPT-2 (2019): 1.5B parameters
- GPT-3 (2020): 175B parameters
- Current frontier: 1 Trillion+ parameters
- State-of-the-art results correlate with scale
- Training these models poses fundamental challenges



Challenge #1 - The Memory Wall

- **Core constraint**: Modern large models cannot fit on even the largest GPUs (80GB A100)
- GPT-3 memory requirements (calculated, not from paper):
- **Weights**: 350GB | Gradients: 350GB | Optimizer states: 700GB | Activations: variable
- **Total**: ~1.4TB needed
- Available: 80GB
- Implication: Model parallelism is physically required, not just an optimization

Challenge #2 - The Compute Wall

- Single GPU baseline: Training GPT-3 would take 288 years on a single V100
- Even with parallelism: Unrealistically long training times without careful scaling
- Research requirement: ~1 month for practical iteration
- The gap: Need 100-1000× speedup through parallelization
- But: Naive parallelization hits efficiency limitations at scale

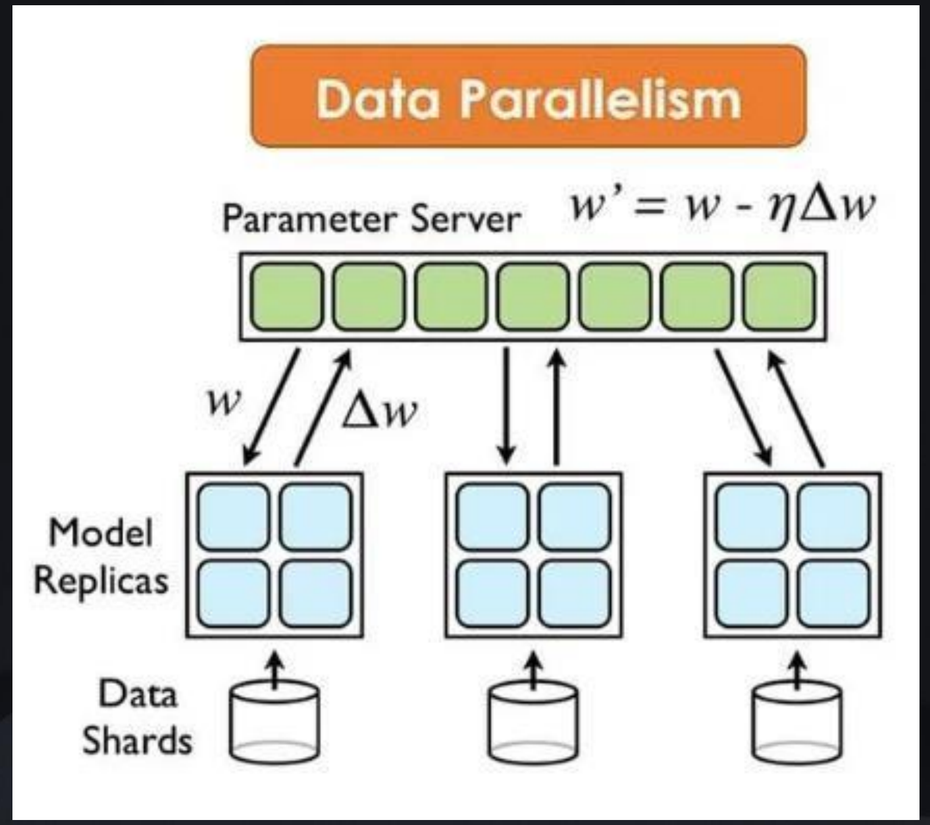
Data Parallelism - The Standard Approach

Approach: Replicate entire model on each GPU

Split: Different data batches to each GPU

Synchronization: All-reduce gradients across GPUs

Advantages:
Simple to implement
Perfect scaling in ideal conditions
Minimal code changes
Communication: All-reduce of gradients (size = model parameters)



Where Data Parallelism Breaks Down

Limitation #1: Memory Wall (The Hard Limit)
Model must fit on a single GPU
GPT-3 (175B): Needs ~1.4TB → Cannot use data parallelism alone
Larger models: Physically impossible, not just inefficient

Limitation #2: Scaling Ceiling (The Soft Limit)

- Maximum parallelism = global batch size
- Example: Batch size 1024 → max 1024 GPUs
- Beyond this: per-GPU batch size < 1 → training breaks

Limitation #3: Communication Overhead
All-reduce cost ∝ model size
Per-GPU batch size ↓ → communication/computation ratio ↑
Below critical per-GPU batch size: spend more time communicating than computing

Model Parallelism - The Necessity

- Core idea: Split the model itself across multiple devices
- Breaks the single-GPU memory constraint
- Enables training models larger than any GPU
- Two complementary approaches:
- Tensor Parallelism (Intra-layer): Split operations within a layer
- Pipeline Parallelism (Inter-layer): Split layers across devices
- Trade-off: Adds communication, complexity vs. enables scale

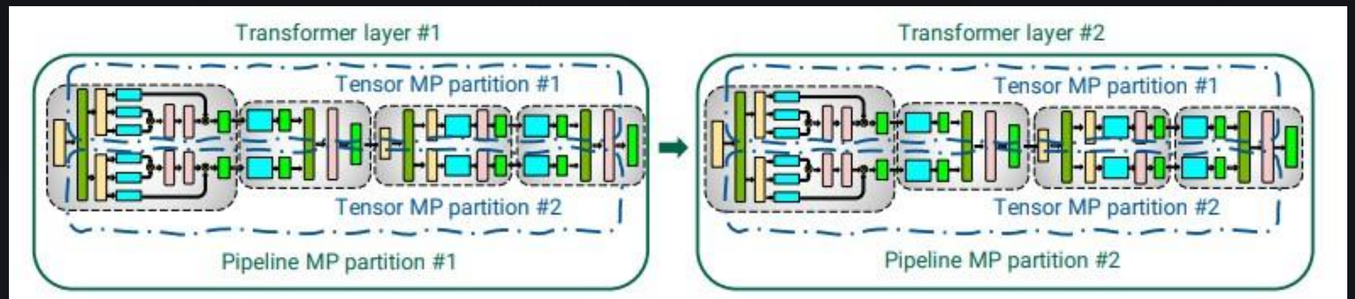


Figure 2: Combination of tensor and pipeline model parallelism (MP) used in this work for transformer-based models.

Tensor Parallelism (Intra-layer)

- Granularity: Within a single layer (horizontal partitioning)
- Example: In MLP block, split W_1 by columns, W_2 by rows across GPUs
- In self-attention: Split Q, K, V matrices across attention heads
- Communication: All-reduce operations per layer (high frequency, small messages)
- Bandwidth requirement: Needs extremely fast interconnect (NVLink within node)
- Limitation: Doesn't scale well across nodes (requires high-bandwidth links)
- Pattern: 2 all-reduces per layer per forward pass

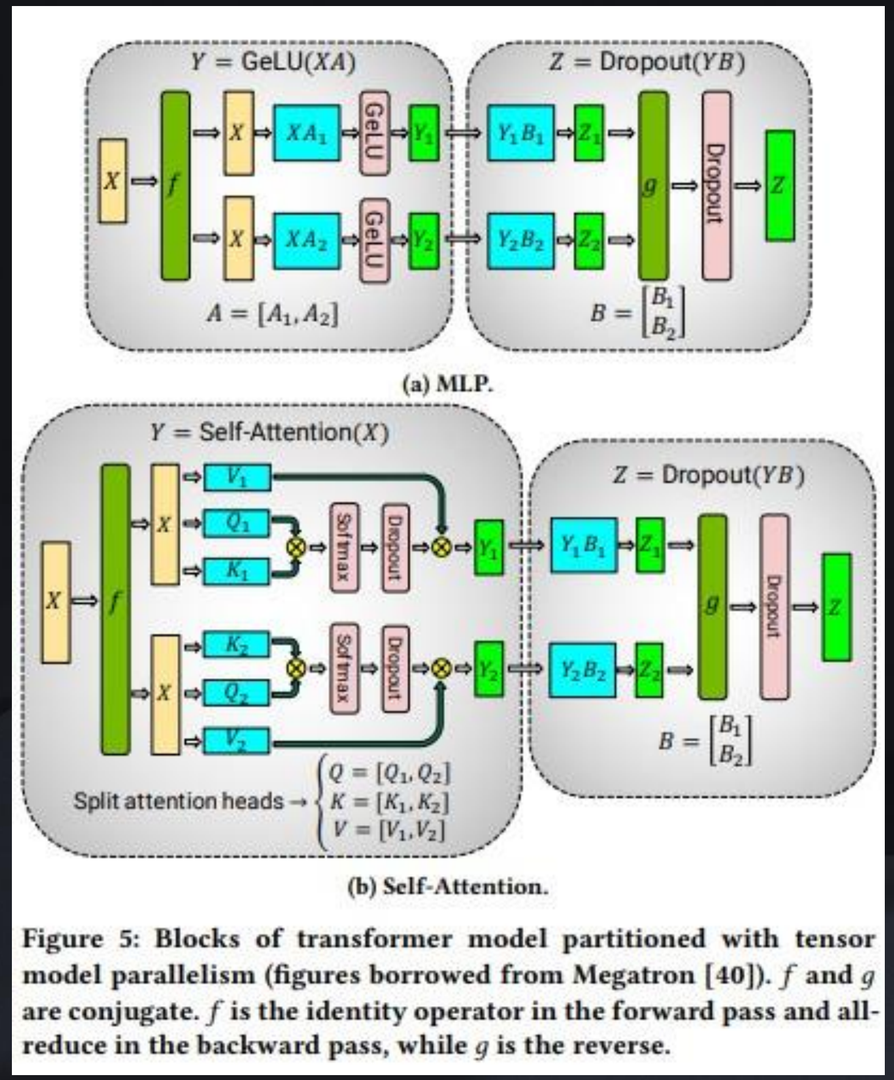
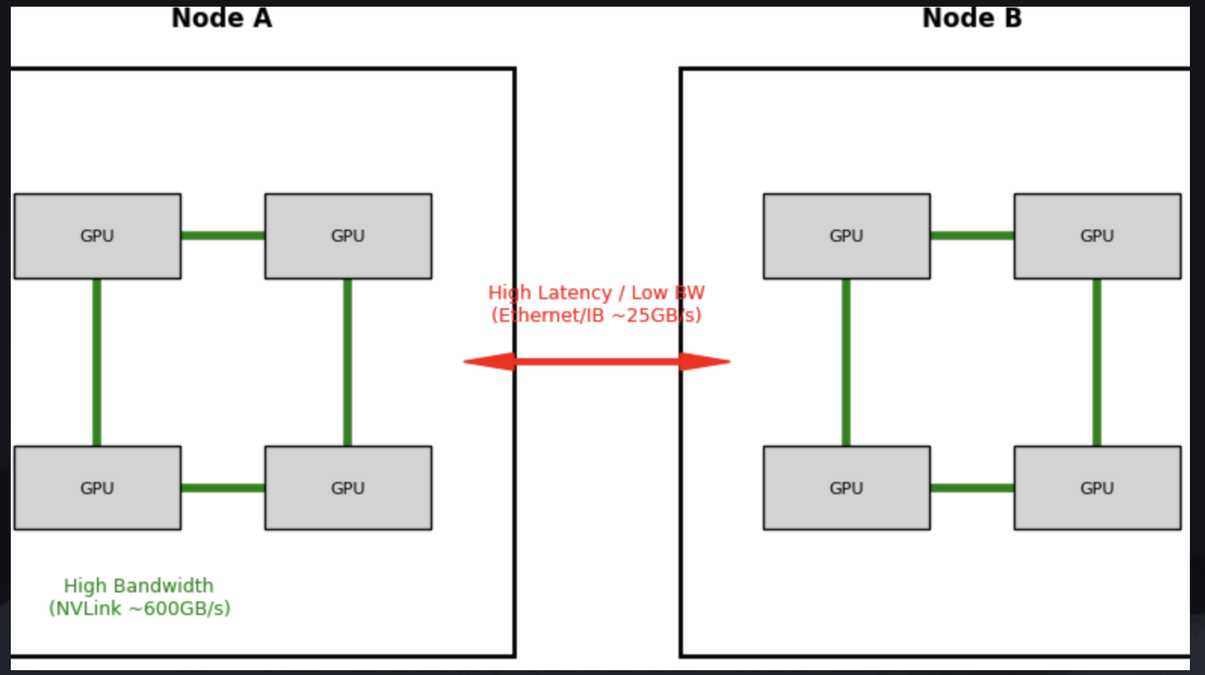


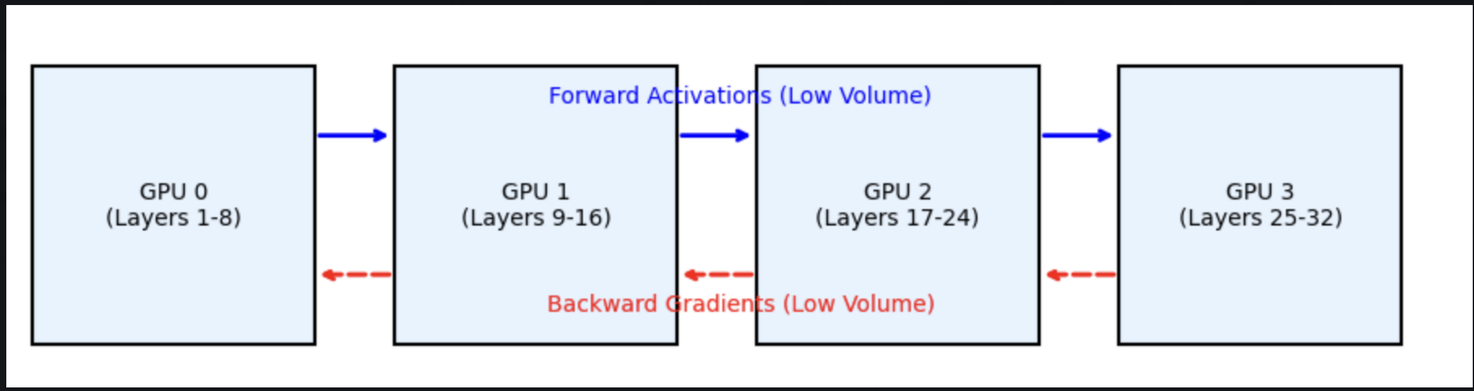
Figure 5: Blocks of transformer model partitioned with tensor model parallelism (figures borrowed from Megatron [40]). f and g are conjugate. f is the identity operator in the forward pass and all-reduce in the backward pass, while g is the reverse.

Why Tensor Parallelism Across Nodes Is Inefficient?

- Tensor parallelism: splits individual matrix operations across devices
- Synchronization cost: requires frequent all-reduce during forward and backward passes
- Interconnect mismatch: fast intra-node links (e.g., NVLink) vs. slower inter-node links (e.g., InfiniBand)
- Scaling issue: computation repeatedly stalls waiting for cross-node communication
- Result: communication dominates compute at scale, limiting efficiency

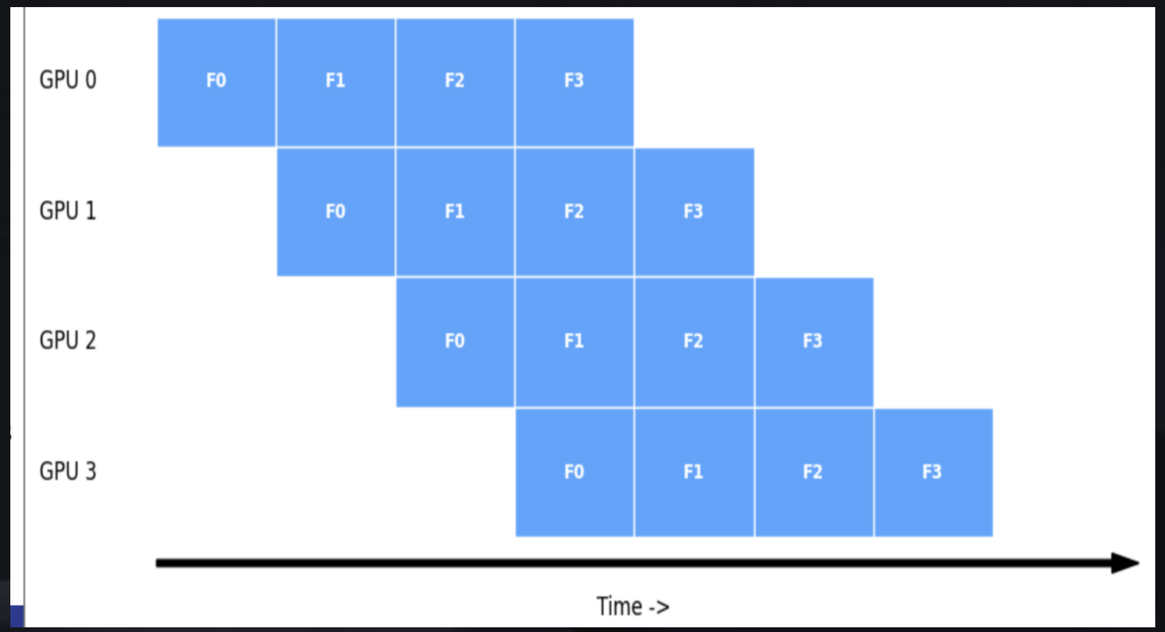


Pipeline Parallelism: High-Level Idea



- Layer-wise partitioning: split the model vertically across devices
- Stage ownership: each device holds a contiguous block of layers
- Execution flow: forward activations move to the next stage; backward gradients move to the previous stage
- Communication: point-to-point send/recv (not collective all-reduce)
- Scalability: communication volume depends on hidden size, not total model parameters
- Benefit: more efficient than tensor parallelism for cross-node training

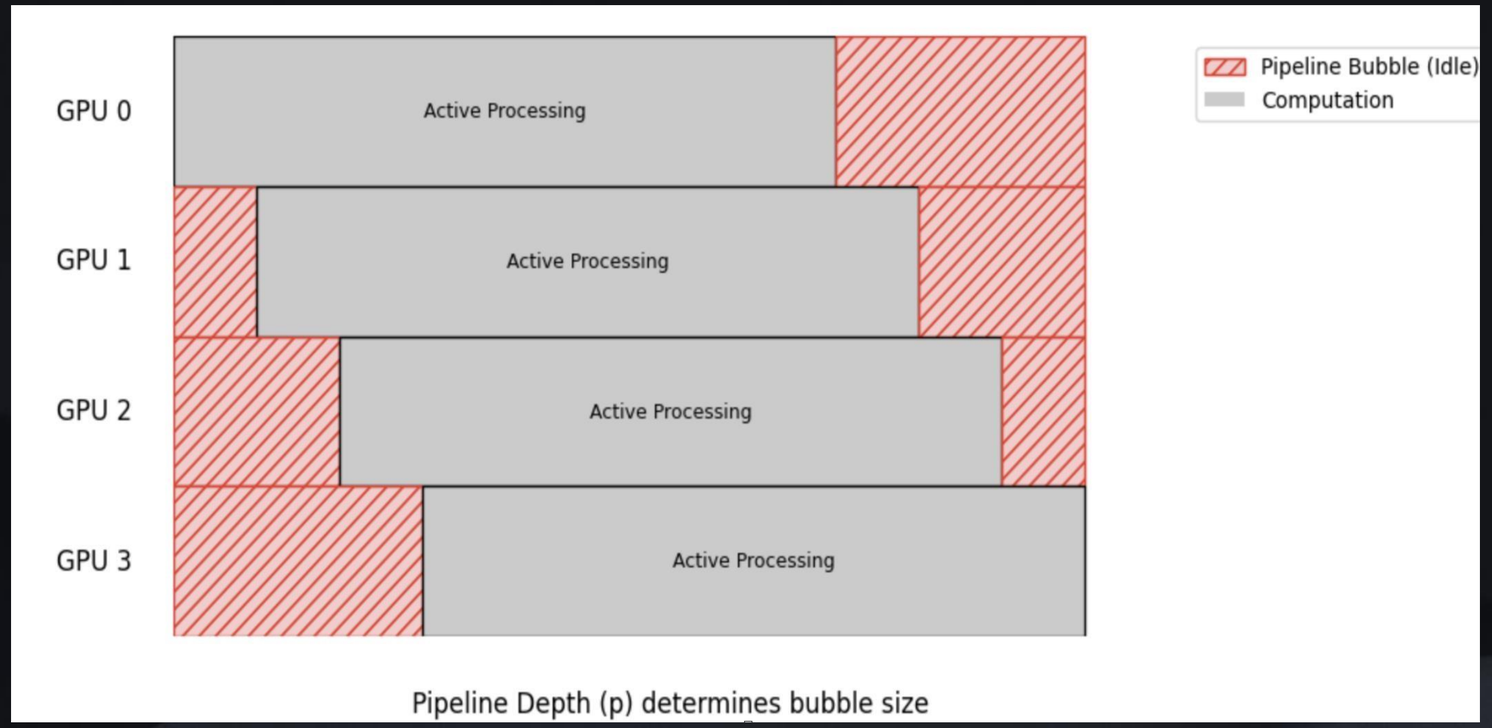
Why is Microbatching Required?



- Naive batching: a single large batch forces pipeline stages to execute sequentially
- Microbatching: split the global batch into smaller microbatches
- Pipeline filling: enables multiple microbatches to be in flight simultaneously
- Concurrency: different pipeline stages can work at the same time
- Outcome: significantly improves GPU utilization

Why Pipeline Bubbles Still Exist

- Pipeline fill: early stages start first while later stages wait for inputs
- Pipeline drain: later stages finish last while earlier stages become idle
- Synchronous training: all microbatches must complete before the optimizer step
- Idle regions: the shaded areas in the timeline represent unused GPU time
- Scaling effect: deeper pipelines lead to larger bubbles and lower efficiency



Why Are Bubbles Fundamentally Hard to Eliminate?

- Pipeline bubbles are not caused by poor implementation
- They arise from batch-level synchronization constraints
- All microbatches in a batch must
  - use the same weights
  - complete forward and backward passes
- Optimizer update enforces a global barrier
- Increasing microbatches reduces bubbles, but:
  - increases memory increases latency
  - does not eliminate bubbles entirely

Optimizing Parallelization Configurations

- Parallelization Dimensions
  - $\boldsymbol{p}$ - pipeline parallelism
  - $\boldsymbol{t}$ - tensor parallelism
  - $\boldsymbol{d}$ - model parallelism
- Constrained to n GPUs: $\boldsymbol{n = p \times t \times d}$
- Memory/Model Constraint: $\boldsymbol{p \times t}$ GPUs must be able to hold the model.
- Microbatch size ($\boldsymbol{b}$), $\boldsymbol{p}$, $\boldsymbol{t}$, and $\boldsymbol{d}$ all change the pipeline bubble size.
- **How do we pick these 4 values?**

Pipeline Bubble Size Model

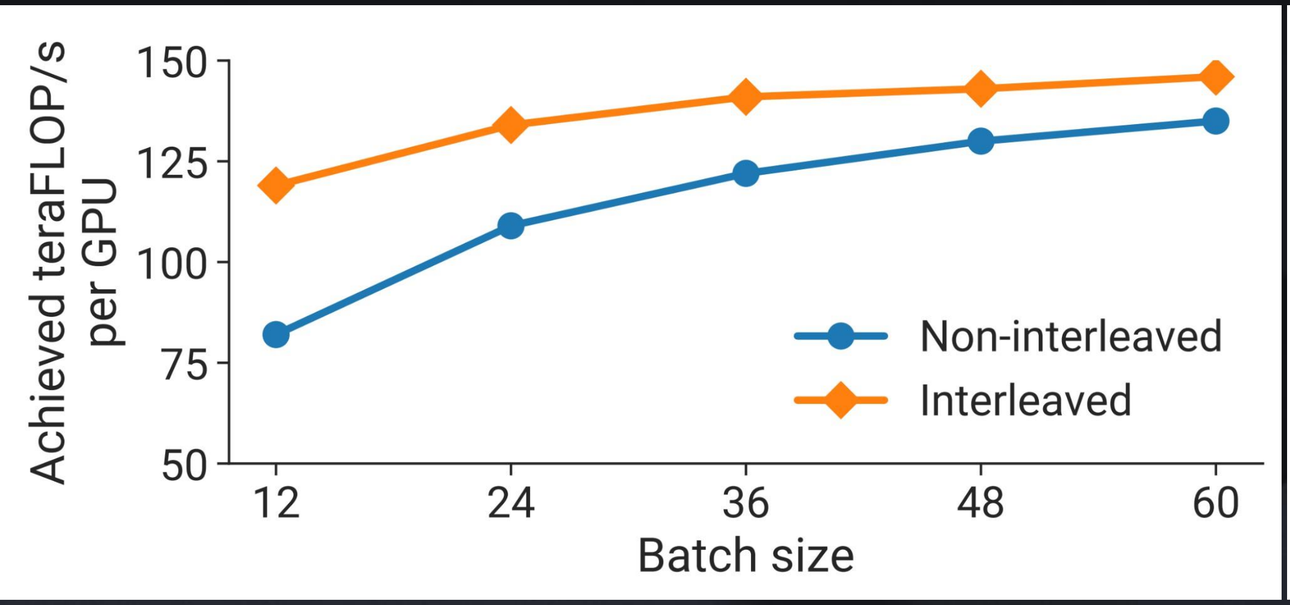
- Let $\boldsymbol{B}$ = batch size, $\boldsymbol{b}$ = microbatch size.
- Let $\boldsymbol{m} = \boldsymbol{B} \div (\boldsymbol{b} \times \boldsymbol{d})$ = microbatches/batch/pipeline.

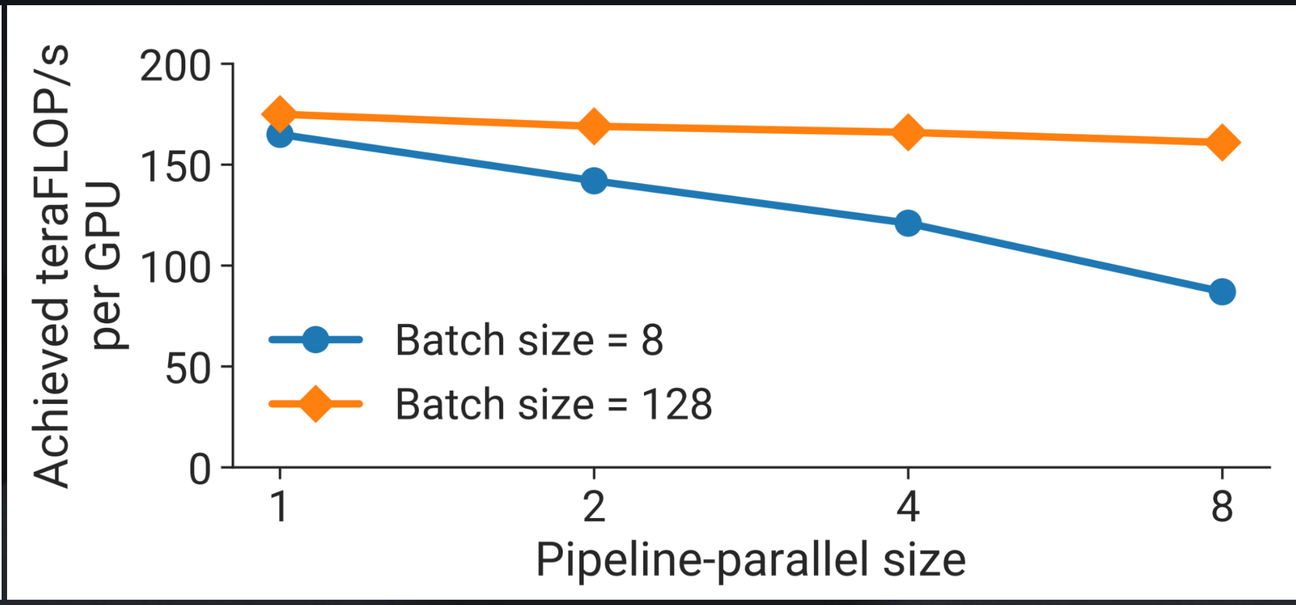
$$\frac{t_{pb}}{t_{id}} = \frac{p-1}{m}$$

- $\boldsymbol{p} = \boldsymbol{n} \div (\boldsymbol{t} \times \boldsymbol{d})$
- Higher $\boldsymbol{t}$, $\boldsymbol{d}$, or $\boldsymbol{B}$ reduces bubble size!

Larger Bubbles Reduce FLOP/s

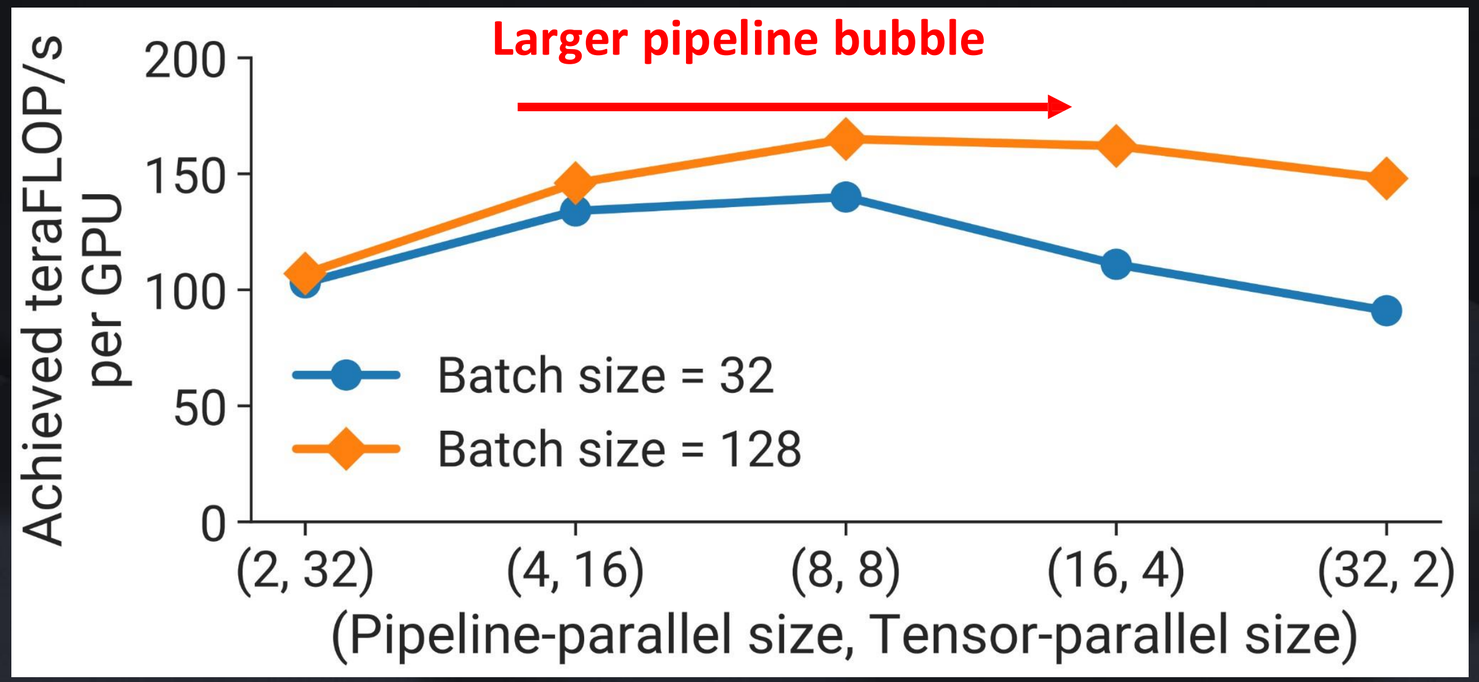
- Larger batch size reduces the size of the bubble, and increases FLOP/s.
- Pipeline parallelism increases the size of the bubble, and decreases FLOP/s.





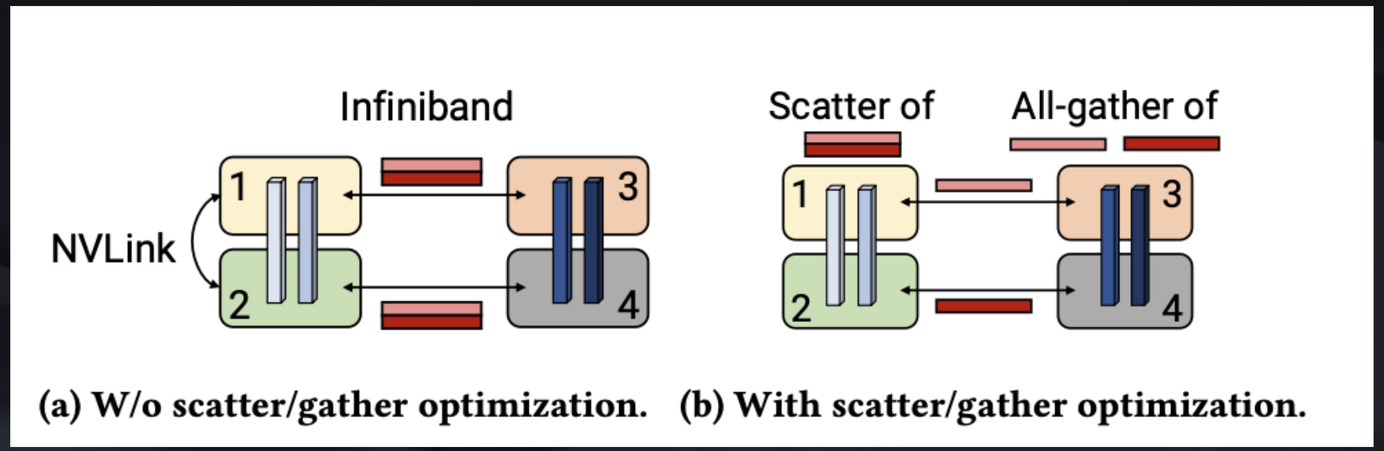
The Tensor Parallelism Limit

- Tensors need to be all-reduced multiple times for the forward and backward pass. Thus, tensor parallelism has higher communication than pipeline.
- NVLink **within a GPU node** make high communication reasonable.



Scatter/Gather Communication Optimization

- NVLink in a single server of GPUs
- Infiniband across multiple servers
- Reduce the overhead of cross-node communication by transporting smaller data chunks

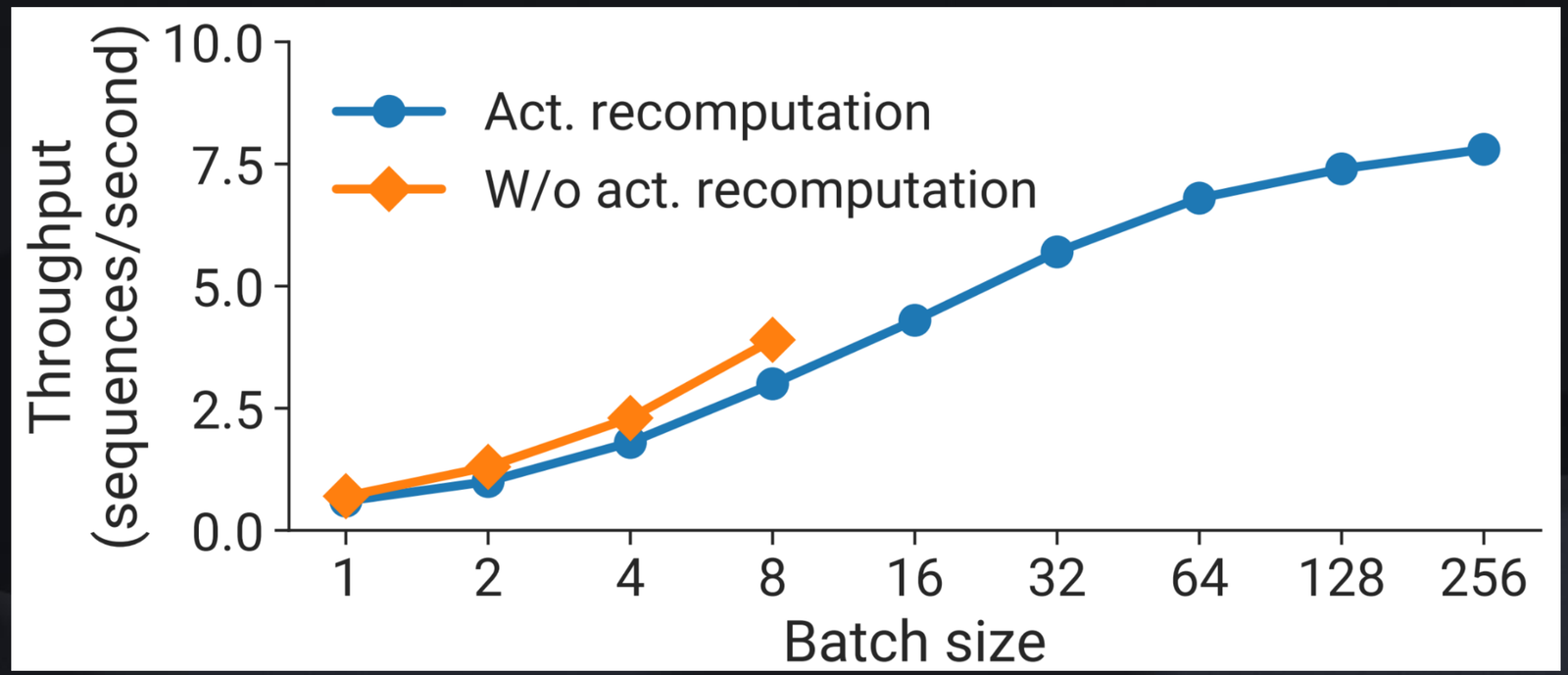


Computation Optimization

- New data layout in the transformer layer
  - Avoid memory-intensive transpose operations; Use strided batched GEMM kernels
- Fused Kernels
  - Element-wise operations
  - Eg. bias + GeLU, bias + dropout + add
- Custom Kernels
  - Enable the fusion of scale, mask, and softmax (reduction) operations
  - General masking & Implicit causal masking

Final Optimizations

- Best microbatch size is model dependent; no general prescription.
  - Larger microbatch size → fewer microbatches (m) → larger bubble.
  - Although, larger microbatch size can give higher arithmetic intensity for kernels.
- Activation can be recomputed instead of stored to reduce memory footprint.



Limitations

- This paper only discusses heuristic methods for **manual** tuning.
- No good method for consistently selecting microbatch sizes.
- Fixed 3D parameter tuning that doesn’t adapt to complex models or clusters.

Key Takeaways

- Not enough GPU memory to increase $\boldsymbol{d}$ to $\boldsymbol{n}$ for maximum data parallelism.
- Use as much tensor parallelism as possible **within each node** to fit the model.
- Increase pipeline parallelism just enough to fit the model.
- Scale data parallelism to the rest of the GPUs.
- Prefer larger batch sizes.

Let us now dive into Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning

The Problem with Manual Parallelism Plans

Manual plans (e.g., Megatron-LM's 3D Parallelism) assume uniform, repeated layers

- Equal layers per stage, same operator/data parallelism config for all layers
- Cannot generalize to heterogeneous models (e.g., MoE, Wide-ResNet) or different clusters

Existing auto-parallelization systems only explore one model parallelism dimension

- Tofu, FlexFlow: intra-operator only, no pipeline
- DAPPLE, PipeDream: pipeline + data only, no operator parallelism

The full joint search space is combinatorially explosive and intractable to explore naively

Alpa's Key Insight — A New View of Parallelisms

- Alpa re-categorizes all parallelism into just two orthogonal levels:
- Intra-operator: partitions tensors (subsumes data, operator, and ZeRO parallelism)
- Inter-operator: assigns whole operators to different devices (i.e., pipeline parallelism)
- Maps to cluster hierarchy: intra-op on fast NVLink, inter-op on slower InfiniBand
- Hierarchical optimization: DP for inter-op stage slicing, ILP for intra-op sharding
- First compiler to automatically cover the full parallelism space

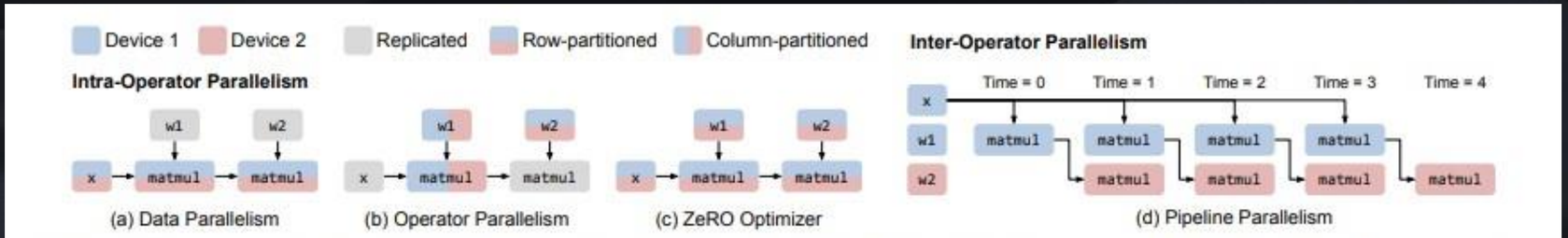


Figure 2: Common parallelization techniques for training a 2-layer Multi-layer Perceptron (MLP). Only the forward pass is shown. “x” is the input data. “w1” and “w2” are two weight matrices.

Alpa Overview

- Alpa is a compiler with 3 novel compilation passes.
  1. Inter-Op Pass: assign stages to meshes. (DP)
  2. Intra-Op Pass: optimize parallel execution of a stage within a mesh. (ILP)
  3. Runtime Orchestration: compile the model to the calculated shard assignments.
- Alpa is invoked by the developer simply with the **@parallelize** python declarator.

The Intra-Op Pass

Sharding Spec

- Notation for representing the sharding distributions of each tensor within a mesh. **R**: replicate, **S**: shard.
- When an operator's output spec doesn't match the next operator's input spec, the tensor needs to be resharded.
  - This cost is modelled directly.

2D Tensor Sharding Specs on 2 x 2 Mesh

Spec	Device 0	Device 1	Device 2	Device 3
RR	$A[0:N, 0:M]$	$A[0:N, 0:M]$	$A[0:N, 0:M]$	$A[0:N, 0:M]$
S^0S^1	$A[0:\frac{N}{2}, 0:\frac{M}{2}]$	$A[0:\frac{N}{2}, \frac{M}{2}:M]$	$A[\frac{N}{2}:N, 0:\frac{M}{2}]$	$A[\frac{N}{2}:N, \frac{M}{2}:M]$
S^1S^0	$A[0:\frac{N}{2}, 0:\frac{M}{2}]$	$A[\frac{N}{2}:N, 0:\frac{M}{2}]$	$A[0:\frac{N}{2}, \frac{M}{2}:M]$	$A[\frac{N}{2}:N, \frac{M}{2}:M]$
S^0R	$A[0:\frac{N}{2}, 0:M]$	$A[0:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:N, 0:M]$	$A[\frac{N}{2}:N, 0:M]$
S^1R	$A[0:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:N, 0:M]$	$A[0:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:N, 0:M]$
RS^0	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:M]$	$A[0:N, \frac{M}{2}:M]$
RS^1	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:M]$	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:M]$
$S^{01}R$	$A[0:\frac{N}{4}, 0:M]$	$A[\frac{N}{4}:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:\frac{3N}{4}, 0:M]$	$A[\frac{3N}{4}:N, 0:M]$
RS^{01}	$A[0:N, 0:\frac{M}{4}]$	$A[0:N, \frac{M}{4}:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:\frac{3M}{4}]$	$A[0:N, \frac{3M}{4}:M]$

The Intra-Op Pass

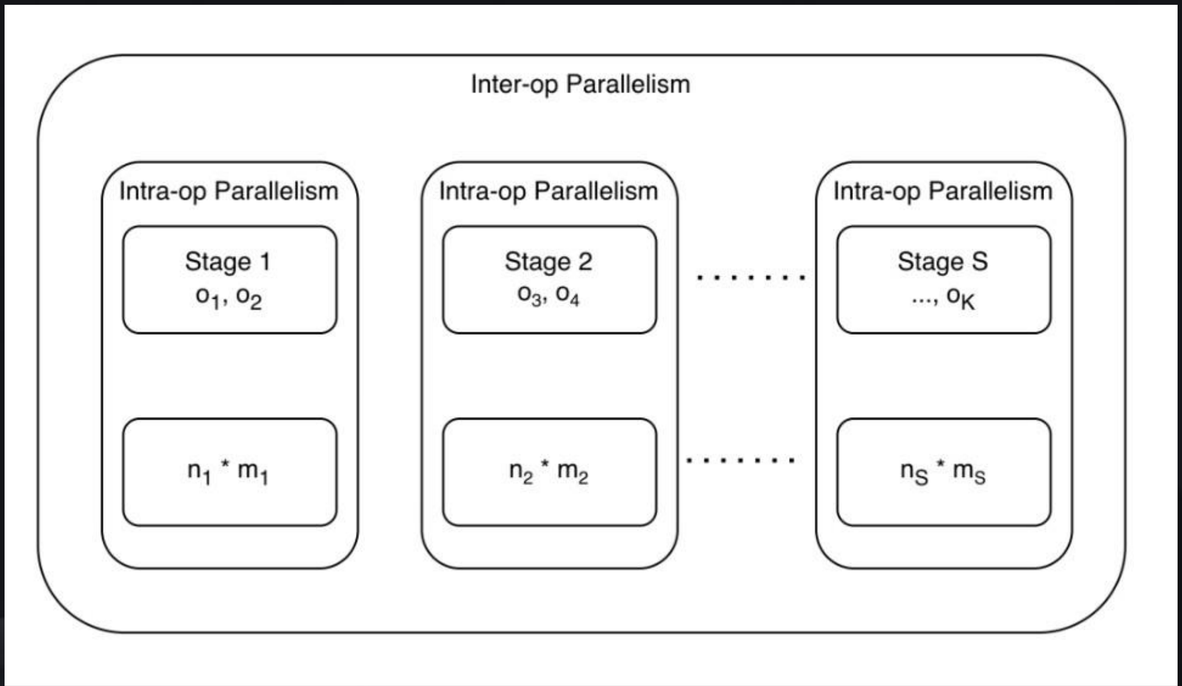
Algorithm Selection and Cost Optimization

- Each of 80 primitive operations (e.g. matmul) have different algorithms with different costs and required input shapes.
- A given selection of operator algorithms has a computation cost and a communication cost.
- Search through possible assignments with off-the-shelf integer linear programming.

3D Matmul Communication Costs

#	Parallel Mapping	Output Spec	Input Specs	Communication Cost
1	$i \to 0, j \to 1$	RS^0S^1	RS^0R, RRS^1	0
2	$i \to 0, k \to 1$	RS^0R	RS^0S^1, RS^1R	$all\text{-}reduce(\frac{M}{n_0}, 1)$
3	$j \to 0, k \to 1$	RRS^0	RRS^1, RS^1S^0	$all\text{-}reduce(\frac{M}{n_0}, 1)$
4	$b \to 0, i \to 1$	S^0S^1R	S^0S^1R, S^0RR	0
5	$b \to 0, k \to 1$	S^0RR	S^0RS^1, S^0S^1R	$all\text{-}reduce(\frac{M}{n_0}, 1)$
6	$i \to \{0,1\}$	$RS^{01}R$	$RS^{01}R, RRR$	0
7	$k \to \{0,1\}$	RRR	$RRS^{01}, RS^{01}R$	$all\text{-}reduce(M, \{0,1\})$

Inter-Operator Parallelism



Computational Gra

o_1, …, o_K Device Clu Mesh(n_1, m_1), …, Mesh(n_S, m_S)
where $\sum_{i=1}^{S} n_i \cdot$

The Space for Inter-Operato arallelism

$$T^* = \min_{\substack{s_1,\ldots,s_S;\\ (n_1,m_1),\ldots,(n_S,m_S)}} \left\{ \sum_{i=1}^{S} t_i + (B-1) \cdot \max_{1 \le j \le S} \{t_j\} \right\}$$

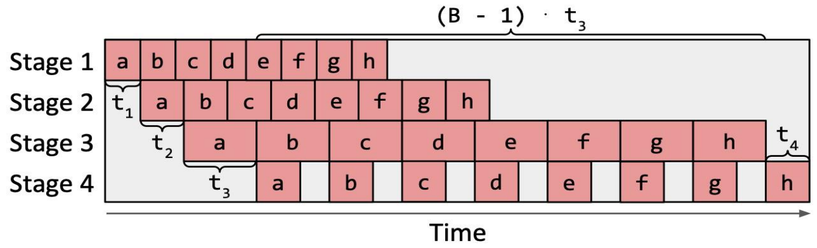


Figure 5: Illustration of the total latency of a pipeline, which is determined by two parts: the total latency of all stages ($t_1 + t_2 + t_3 + t_4$) and the latency of the slowest stage ($(B-1) \cdot t_3$).

- B: Numbe icrobatches
- S: Numbe ages
- $t_i = t_{intra}$ (s_i h(n_i , m_i))
- Forward o d Backward op same submesh
  - Reduce unication cost
- . $\sum_{i=1}^{S} n_i \cdot$
  - Use up all available compute resources

DP Formulation

- Before DP: Reduce search space => (n, m): (1, 1), (1, 2), (1, 4) . . . (1, 2^m) and (2,M),(3,M),...,(N,M)
- Boundary case: *F(0, K + 1, 0; t_{max}) = 0*

$$F(s,k,d;t_{max}) = \min_{\substack{k\le i\le K \\ n_s\cdot m_s\le d}} \left\{ \begin{array}{l} t_{intra}((o_k,\ldots,o_i),Mesh(n_s,m_s),s) \\ +F(s-1,i+1,d-n_s\cdot m_s;t_{max}) \\ \mid t_{intra}((o_k,\ldots,o_i),Mesh(n_s,m_s),s)\le t_{max} \end{array} \right\}$$

$$T^*(t_{max}) = \min_s\{F(s,0,N\cdot M;t_{max})\} + (B-1)\cdot t_{max}$$

DP Formulation - Optimization

- Early pruning
  - Enumerate t_{max} in an ascending order
  - Quit when $B \bullet t_{max} > T^*$
- Operator clustering
  - Cluster neighboring operators to reduce $K: (o_1,...,o_K) => L: (l_1,...,l_L)$, where $L << K$
  - Rule of merging operators:
    - Little computation (eg. ReLU)
    - Neighboring operators may cause substantial communication

Parallelism Orchestration: Cross-mesh Resharding

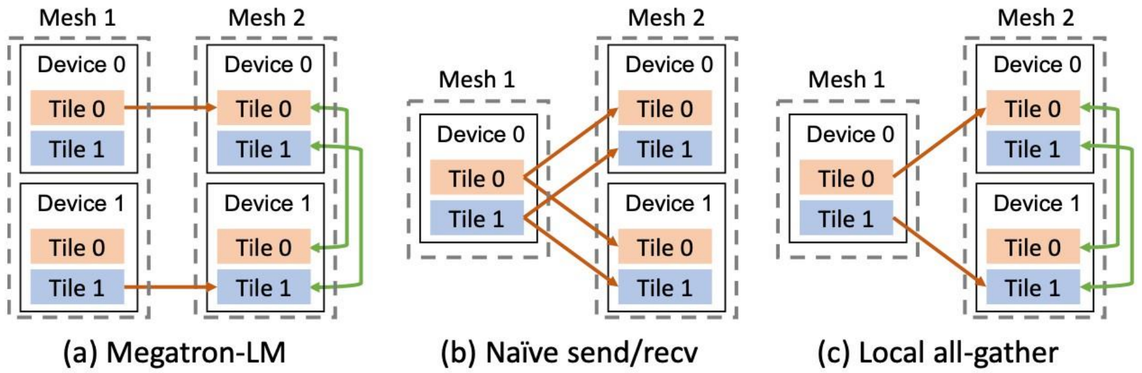


Figure 6: Cross-mesh resharding. Red arrows denote send/recv on slow connections. Green arrows denote all-gather on fast connections. (a) The scatter-gather optimization for equal mesh shapes in Megatron-LM. (b) The naive send/recv for unequal mesh shapes. (c) The generalized local all-gather optimization for unequal mesh shapes.

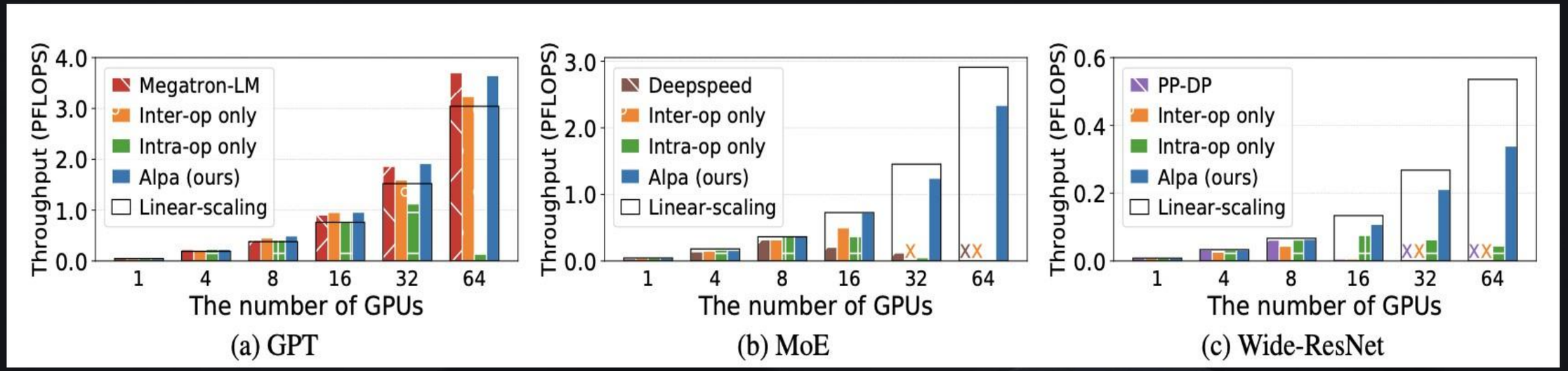
Alpa: local all-gather cross-mesh resharding

- Calculate the correspondences between tensor partitions on the source and destination mesh. (b)
- Replace duplicate sending with higher bandwidth communication. (c)

Parallelism Orchestration: Generating pipeline execution instructions

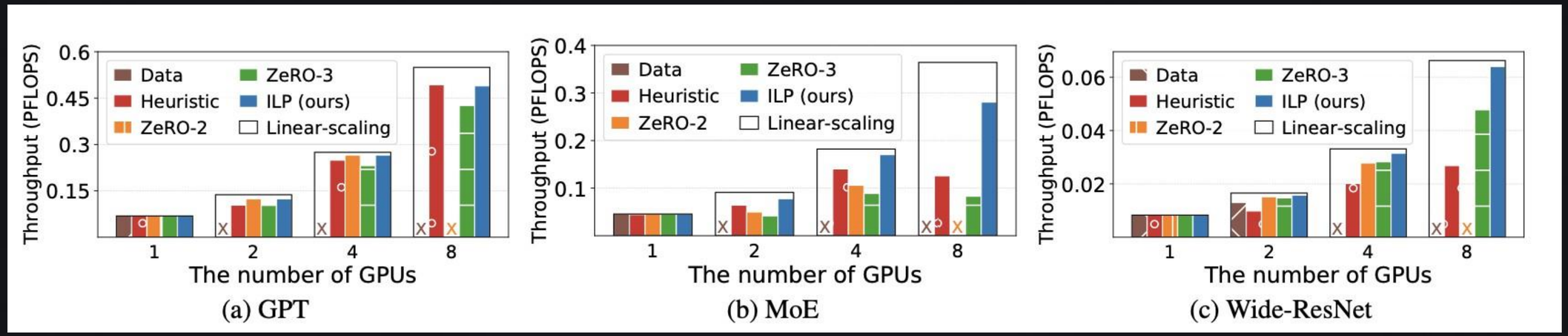
- Static execution instructions
  - Allocation and Deallocation memory
  - Communication between stages following the cross-mesh resharding plan
  - Synchronization
  - Computation
  - Etc.
- MPMD-style runtime

Evaluation: End-to-End Training Throughput



- Alpa achieves **near-optimal throughput** across GPT, MoE, CNN
- Consistently **outperforms manual parallel strategies**
- Benefits grow with **model and cluster scale**

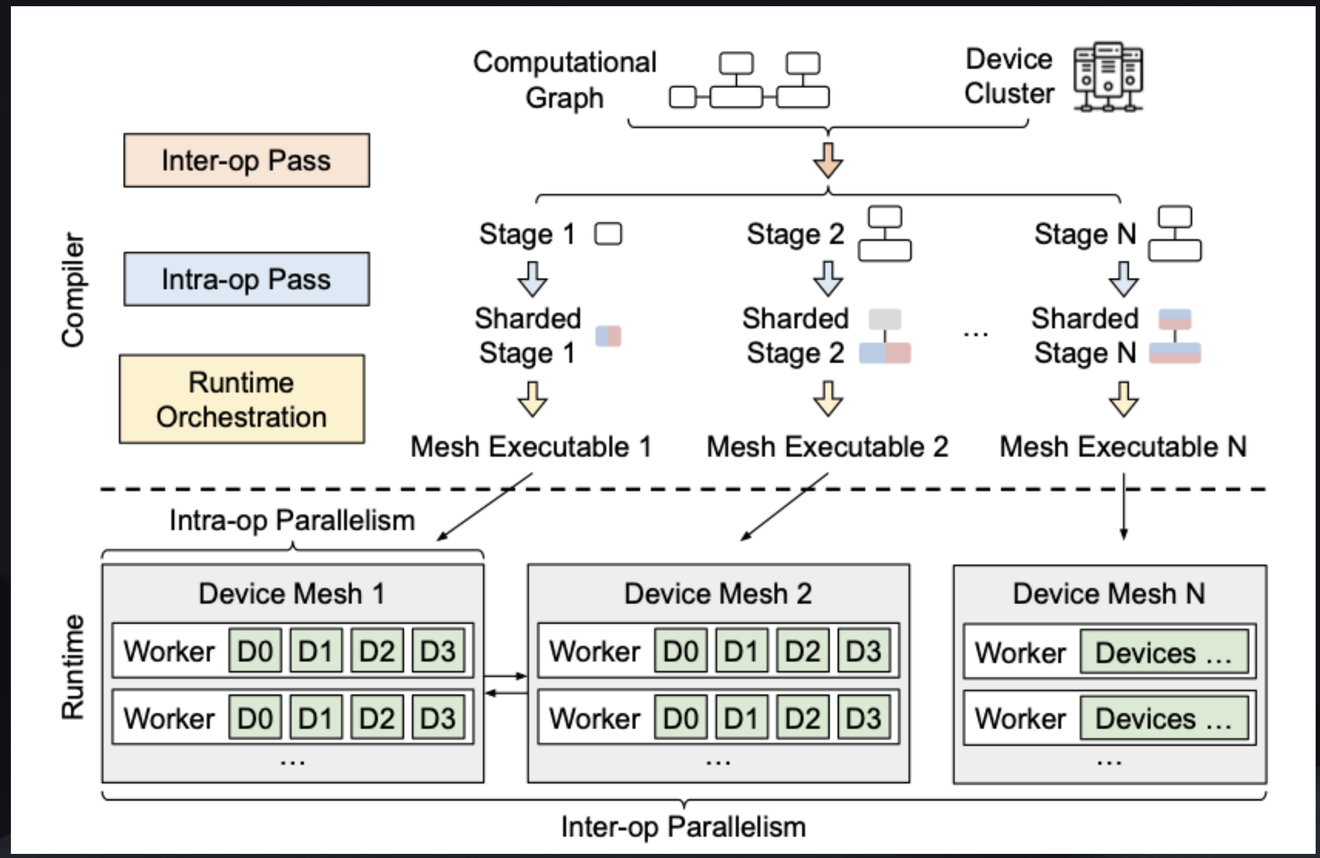
Evaluation: Intra-Operator Parallelism



- Auto-sharding beats **ZeRO and heuristic sharding**
- Better **compute–communication balance**
- Confirms compiler decisions > human tuning

Conclusion: Alpa (Standalone)

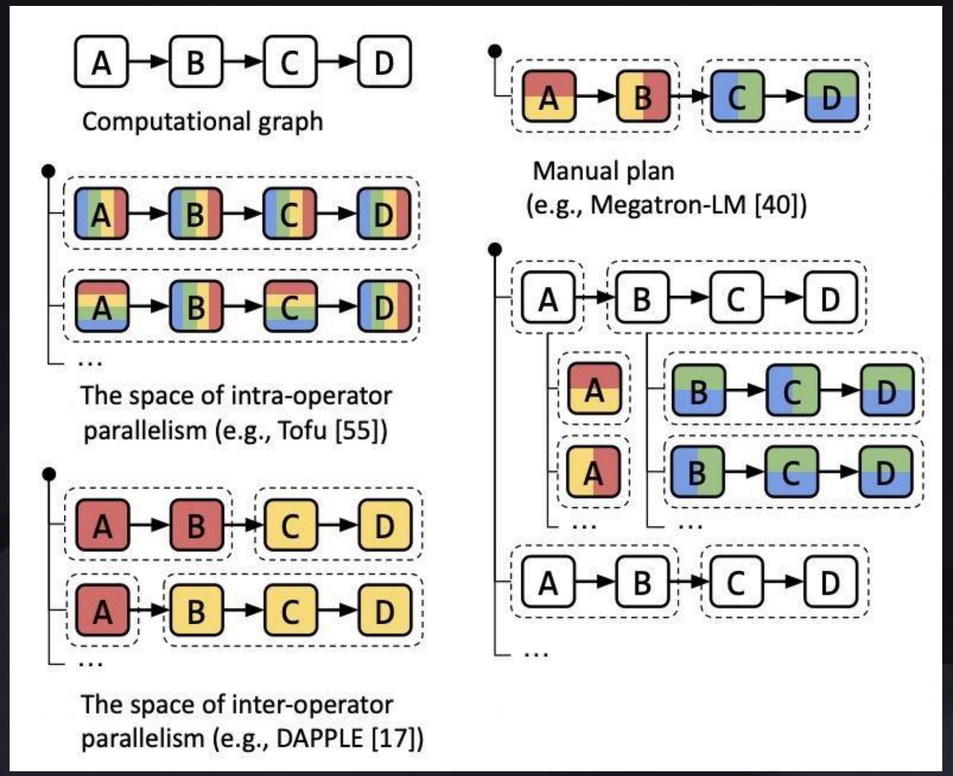
- Unifies **inter-op + intra-op** parallelism
- Compiler-driven plan search with cost modeling
- Removes need for expert-written parallel code



Conclusion: How Alpa Builds on Megatron-LM

Megatron-LM → Alpa: Evolution of Parallelism

- **Megatron-LM**: manual tensor + pipeline parallelism
- **Alpa**: automates and generalizes these ideas
- Compiler replaces human intuition at scale



Thank you all for tuning in :)